

Scripting Languages

MCA First Semester - Mid Semester Examination, January 2023

Question Paper with Model Answers (HTML and CSS)

Course	M.C.A. (First Semester)	Paper Code	TMC-106
Paper	Scripting Languages	Examination	Mid Semester, Jan. 2023
Time	1½ hours	Maximum Marks	50
Pattern	5 questions; attempt any ONE sub-question of each; each sub-question carries 10 marks	References	WHATWG HTML Living Standard, W3C CSS specifications, MDN Web Docs

How to use this document

- Each question from the paper is reproduced with the code/data written inline. **Both options (a) and (b) are answered**, so that the whole HTML/CSS syllabus is covered while revising.
- Each answer is written for **10 marks**: definition, syntax, tables of attributes, complete working code, explanation of the output and a short exam tip.
- All code is valid HTML5/CSS3 unless stated. Frames are obsolete in HTML5 but are explained because the question asks for them; the modern replacement is given as well.
- The **Silberschatz** book mentioned in your earlier request is an Operating Systems text and does not cover HTML or CSS, so this paper uses the standard web references listed above. If your syllabus prescribes a particular Scripting Languages textbook, use its terminology for the definitions.
- The last pages contain a revision sheet with tag/property cheat-sheets and practice questions for the rest of the syllabus.

Contents

Question 1	2
Question 2	8
Question 3	13
Question 4	18
Question 5	24
Quick Revision Sheet	28

Question	Topics covered
Q1	(a) <map> tag and image-map shapes; (b) metadata and the <meta> tag
Q2	(a) CSS and its types - inline, internal, external; (b) active-link colour, square list style, background image
Q3	(a) five paragraph classes with background, colour, margin, padding, border; (b) CSS pseudo-classes
Q4	(a) <frame> tag and <input> attributes; (b) forms, form attributes and components
Q5	(a) three hyperlinks opening in a new window; (b) CSS box model

Question 1

10 Marks | Attempt any ONE sub-question in the exam | CO1 | Both options are answered below for revision

Q1 (a) [10 Marks, CO1]

Explain <MAP> tag and different shapes used in image map with suitable example.

1. What is an image map?

An **image map** is an image in which **different regions (hot spots)** act as separate hyperlinks. Clicking on one part of the picture takes the user to one page, and clicking on another part takes the user to a different page. Examples: a map of India where each state links to its own page, a picture of a building where each department is clickable, a navigation banner with several buttons drawn in one image. Instead of cutting the picture into many small images, one image is used and the hot spots are described with the <map> and <area> tags.

2. The <map> tag

The <map> tag **defines a client-side image map**. It is a container that holds one or more <area> elements, each describing one clickable region of the image. The map is connected to an image with the **usemap** attribute of the tag. The <map> element itself is not displayed in the browser.

- **Syntax:** <map name="mapname"> <area ...> <area ...> </map> (needs an opening and a closing tag; <area> is an empty tag).
- The **name** attribute of <map> is **required**; its value must match the value of the **usemap** attribute of the image, written with a leading # usemap="#mapname".
- The <map> can be placed anywhere inside the <body> (before or after the image) and can be used with or <object>.

Attributes of and <map> used for image maps

Tag	Attribute	Purpose
	usemap	Value #name - links the image to the <map> having the same name.
	ismap	Marks the image as a server-side image map (coordinates of the click are sent to the server).
	src, alt, width, height	Normal image attributes; fixed width/height keep the pixel coordinates correct.
<map>	name	Name of the map (required); referred to by usemap.
<map>	id	Optional; if both are used, name and id must have the same value.

3. The <area> tag and its attributes

Each <area> defines **one hot spot**: its shape, its position (coordinates) and the link it opens.

Attribute	Meaning	Values / example
shape	Shape of the hot spot.	rect, circle, poly, default
coords	Pixel coordinates that define the shape; origin (0,0) is the top-left corner of the image.	coords="20,20,120,80"
href	URL opened when the region is clicked.	href="page.html"
alt	Alternate text (required when href is present) for screen readers and text browsers.	alt="Rectangle"
title	Tool-tip shown on hover.	title="Go to page"

Attribute	Meaning	Values / example
target	Where to open the link.	_self, _blank, _parent, _top
download, rel	Download the target / relationship of the link.	download, rel="noopener"
noreferrer	Region without any link (obsolete in HTML5; omit href instead).	-

4. Shapes used in an image map

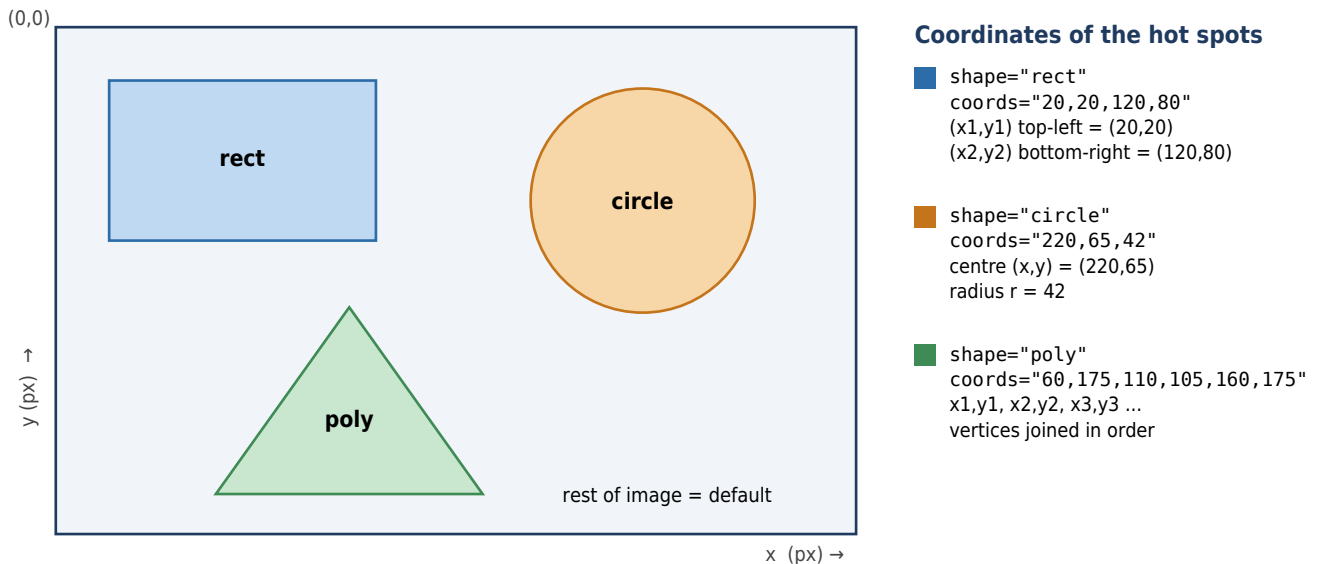


Fig. 1 - Rectangle, circle and polygon hot spots; the coordinate origin is the top-left corner of the image

Shape	Meaning	coords format	Example
rect	Rectangle (also square).	x1,y1,x2,y2 : top-left corner and bottom-right corner.	coords="20,20,120,80"
circle	Circle.	x,y,r : centre coordinates and radius (in pixels).	coords="220,65,42"
poly	Polygon with any number of sides (triangle, star, irregular map region).	x1,y1,x2,y2,...,xn,yn : all vertices in order; the last point is joined to the first automatically.	coords="60,175,110,105,160,175"
default	The whole remaining area of the image not covered by other shapes.	No coords needed.	shape="default"

5. Complete example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Image Map Example</title>
</head>
<body>
  <h2>Click on a shape</h2>
  <!-- the image is connected to the map through usemap="#shapemap" -->
  

  <map name="shapemap">
    <area shape="rect" coords="20,20,120,80" href="rectangle.html" alt="Rectangle"
      title="Rectangle">
  </map>
```

```

<area shape="circle" coords="220,65,42" href="circle.html" alt="Circle"
      title="Circle">
<area shape="poly" coords="60,175,110,105,160,175" href="triangle.html"
      alt="Triangle" target="_blank">
<area shape="default" href="home.html" alt="Rest of the picture">
</map>
</body>
</html>

```

Working: when the user clicks inside the rectangle the browser opens *rectangle.html*; inside the circle it opens *circle.html*; inside the triangle it opens *triangle.html* in a new tab; anywhere else *home.html* opens. The pointer changes to a hand over every hot spot. If two areas overlap, the **first one in the source order** is used.

6. Client-side vs server-side image maps

Basis	Client-side image map	Server-side image map
Tags used	, <map>, <area>	<a> and a program on the server.
Who processes the click?	The browser decides which region was clicked.	The click position (x,y) is sent to the server (?x,y) which decides the page.
Speed	Fast; no server round trip.	Slower; needs a request to the server.
Link visibility	Destination URL is shown in the status bar on hover.	Only coordinates are shown.
Works without server programs?	Yes.	No; needs CGI/PHP or similar.
Use today	Standard method.	Rare; mostly obsolete.

7. Advantages and disadvantages

Advantages	Disadvantages
Many links from a single image; less HTTP requests than many small images.	Coordinates are fixed pixels; the map does not scale automatically when the image is resized (responsive design problem).
Irregular shapes (map regions, body parts, devices) can be made clickable using poly.	Finding coordinates by hand is tedious (use an editor/tool or browser developer tools).
Client-side maps are quick and need no server program.	Text inside the image is not searchable and is less accessible; every area needs alt text.
Areas are keyboard focusable and can carry title tool-tips.	Modern alternatives (CSS positioned links, SVG links) are more flexible and responsive.

Exam tip - key points to remember

- Three parts: , <map name="name">, and one <area> per hot spot.
- Shapes: **rect** = x1,y1,x2,y2; **circle** = x,y,r; **poly** = x1,y1,...,xn,yn; **default** = rest of the image. Origin (0,0) = top-left.

Q1 (b) [10 Marks, CO1]**What is Metadata? Explain with example how Metadata is included in a document?****1. What is metadata?**

Metadata literally means '**data about data**'. It is information that *describes* a document - who created it, what it is about, which character set it uses, when it was changed, which keywords describe it - but it is **not displayed** as part of the page content. In an HTML document, metadata is information about the web page itself that is used by **browsers** (to display the page correctly), **search engines** (to index and describe the page) and **other web services** (social media sites, crawlers, mobile devices). In the same way, a library card describes a book, and the EXIF data of a photograph describes the camera, date and location.

Kinds of metadata

Kind	Describes	Example in a web page
Descriptive	What the document is about.	Title, description, keywords, author.
Structural / technical	How the document is built and how it must be processed.	Character set, viewport, language, link to style sheet.
Administrative	Ownership, rights, dates and handling.	Copyright, publisher, revised date, robots instructions.

2. How metadata is included in an HTML document

Metadata is written inside the **<head> section** of the HTML document, before **<body>**. It is included with the **<meta>** tag, together with some other metadata elements. The **<meta>** tag is an **empty (void) tag** - it has no closing tag - and it carries its information in attributes.

Attributes of the <meta> tag

Attribute	Purpose	Example
charset	Declares the character encoding of the page.	<code><meta charset="UTF-8"></code>
name	Name of the metadata item (with content).	<code><meta name="author" content="Ravi"></code>
content	Value of the metadata item given in name or http-equiv.	<code>content="HTML, CSS tutorial"</code>
http-equiv	Acts like an HTTP response header (refresh, cache, content type, security policy).	<code><meta http-equiv="refresh" content="30"></code>
property	Used by Open Graph (social sharing) metadata.	<code><meta property="og:title" content="..."></code>

Common metadata names

name / item	Function	Example
viewport	Controls layout on mobile screens (responsive design).	<code><meta name="viewport" content="width=device-width, initial-scale=1.0"></code>
description	Short summary of the page; search engines often show it under the title in search results.	<code><meta name="description" content="Free HTML and CSS notes for MCA"></code>
keywords	Comma-separated keywords (major search engines now ignore it).	<code><meta name="keywords" content="HTML, CSS, scripting"></code>
author	Name of the author of the page.	<code><meta name="author" content="A. Sharma"></code>

name / item	Function	Example
robots	Instructions to search-engine crawlers: index/noindex, follow/nofollow.	<meta name="robots" content="index, follow">
generator, theme-color	Software used to create the page; browser toolbar colour on mobile.	<meta name="theme-color" content="#1F3A5F">
refresh	Reload the page or redirect after some seconds.	<meta http-equiv="refresh" content="5; url=home.html">

Other elements that carry document metadata

Element	Role
<title>	Title of the document - shown in the browser tab, bookmarks and search results (required).
<link>	Relationship to external resources: style sheets, icons (favicon), alternate versions, canonical URL.
<base>	Base URL and default target for all relative links of the page.
<style>	Embedded CSS rules for the document.
<script>	Embedded or external script (JavaScript).

3. Example - metadata inside a document

```

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- ----- metadata section ----- -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Notes on HTML and CSS for MCA first semester students">
  <meta name="keywords" content="HTML, CSS, scripting languages, MCA">
  <meta name="author" content="Student Name">
  <meta name="robots" content="index, follow">
  <meta http-equiv="refresh" content="300">           <!-- reload every 5 minutes -->

  <!-- Open Graph data used by social media when the page is shared -->
  <meta property="og:title" content="HTML and CSS Notes">
  <meta property="og:description" content="Revision notes for Scripting Languages">
  <title>Scripting Languages - Notes</title>
  <link rel="stylesheet" href="style.css">
  <link rel="icon" href="favicon.ico">
</head>
<body>
  <h1>Welcome to my notes</h1>
  <p>Only this content is displayed. The metadata above is invisible to the reader.</p>
</body>
</html>

```

Explanation: the browser reads charset to decode the characters correctly, viewport to scale the page on phones, and http-equiv="refresh" to reload the page. A search engine reads description, robots and the <title> to build its search result. A social-media site reads the og: tags to prepare the preview card of a shared link. None of these lines appears inside the page area of the browser.

4. Importance of metadata

1. **Correct display** - character set and viewport make text and layout appear properly on every device.
2. **Search-engine optimisation (SEO)** - title and description attract visitors; robots controls what gets indexed.
3. **Sharing and identification** - Open Graph and Twitter-card tags create rich previews; author and copyright identify ownership.
4. **Automatic actions** - refresh/redirect, caching and security policies (Content-Security-Policy) can be given by http-equiv.
5. **Machine readability** - other programs (crawlers, screen readers, archives) can classify and process the page without reading its whole content. Other file types have metadata too: EXIF in photographs, ID3 tags in MP3 files, document properties in PDF/Word files.

Exam tip - key points to remember

- Metadata = data about data; written in <head> with the void tag <meta> (attributes: charset, name/content, http-equiv, property).
- Give at least four examples: charset, viewport, description, keywords/author, robots, refresh - and show one complete <head> in the answer.

Question 2

10 Marks | Attempt any ONE sub-question in the exam | CO2 | Both options are answered below for revision

Q2 (a) [10 Marks, CO2]

What is CSS? Explain different types of CSS with suitable example also explain advantages and disadvantages of each approach.

1. What is CSS?

CSS (Cascading Style Sheets) is a style-sheet language used to describe the **presentation** (colours, fonts, spacing, layout, backgrounds, borders, animation) of a document written in HTML or XML. HTML defines the *structure and content* of a page, while CSS defines *how that content looks*. CSS is a W3C standard (CSS1 - 1996, CSS2 - 1998, CSS3 and later as modules). The word **cascading** means that when several rules apply to the same element, the browser decides which one wins by a fixed order of priority.

Syntax of a CSS rule

```
selector { property : value ; property : value ; }
```

h1 { color: blue; font-size: 24px; }

		--- value	(what to set)
	+	----- property	(what to change)
+	----- selector		(which HTML element is affected)

{ ... } = declaration block; each property: value; = declaration; /* comment */

Advantages of using CSS

- **Separation of content and presentation** - HTML stays clean, and the design can be changed without touching the content.
- **Reusability and consistency** - one style sheet controls the look of hundreds of pages.
- **Easy maintenance** - a change made in one place is reflected everywhere; saves time and reduces file size (faster downloads through caching).
- **More formatting power** - much richer than HTML attributes (shadows, gradients, transitions, positioning, flexbox/grid).
- **Device independence** - media queries give different layouts for mobile, tablet, desktop and print; also improves accessibility.

2. Types of CSS (ways to add CSS to a page)

(i) Inline CSS

The style is written **inside the HTML tag** using the `style` attribute. It affects **only that one element**.

```
<h1 style="color: red; text-align: center;">Inline styled heading</h1>
<p style="background-color: lightyellow; border: 1px solid gray;">Only this paragraph is styled.</p>
```

(ii) Internal (embedded) CSS

The rules are written inside a `<style>` **element in the <head>** of the same HTML document. They affect **all matching elements of that page only**.


```
<head>
  <style>
    body { background-color: #f0f8ff; font-family: Arial, sans-serif; }
    h1 { color: navy; }
    p { font-size: 16px; line-height: 1.5; }
  </style>
</head>
```

(iii) External CSS

The rules are written in a **separate file with the .css extension** and connected to the page with the `<link>` tag in the `<head>` (or with `@import` inside a style sheet). The same file can be used by any number of pages.

```
<!-- in the HTML file -->
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>

/* style.css */
body { background-color: #f0f8ff; }
h1 { color: navy; text-align: center; }
p { font-size: 16px; }
```

3. Advantages and disadvantages of each type

Type	Advantages	Disadvantages
Inline CSS	Quick way to style one element; no separate file or selector needed; highest priority, so it overrides other rules; useful for testing and for HTML e-mails.	Mixes content with presentation; must be repeated for every element; very hard to maintain; no reuse; cannot style pseudo-classes (hover) or use media queries; larger HTML files.
Internal CSS	Styles the whole page from one place; can use all selectors, classes, ids and pseudo-classes; no extra HTTP request; convenient for a single page.	Applies to one page only, so rules are duplicated on multi-page sites; increases the size of the HTML file; not cached separately.
External CSS	One file styles the entire website; easiest maintenance and consistency; browser caches the file so pages load faster; clean HTML; team members can work separately on design and content.	Page may display unstyled until the CSS file is loaded (extra HTTP request); if the file is missing or the path is wrong the design breaks; not suitable for a small one-page document.

4. Comparison and priority (cascade)

Basis	Inline	Internal	External
Where written	In the style attribute of the tag.	In <code><style></code> inside <code><head></code> .	In a separate .css file.
Scope	One element.	One web page.	Whole website (many pages).
Reusability	None.	Within the page.	Highest.
Maintenance	Difficult.	Moderate.	Easy.
Priority	Highest.	Medium (order matters).	Medium (order matters).
Best used for	A single special case, quick test.	A single page with unique design.	Complete websites.

Cascading order (priority from highest to lowest): (1) rules marked `!important`; (2) **inline** style; (3) **internal or external** style sheet - if both have the same specificity the one that appears *later* in the document wins; (4) the **browser default** style. Inside style sheets the more **specific selector** wins: id (#) > class (.) / pseudo-class > element.

```

<link rel="stylesheet" href="style.css"> /* h1 { color: green; } in the file */
<style> h1 { color: blue; } </style>      /* later, so internal overrides external */
<h1 style="color: red;">Hello</h1>      /* inline wins: the heading is RED */

```

Basic selectors (revision)

Selector	Example	Selects
Element	p { }	All <p> elements.
Class	.note { }	Elements having class="note".
ID	#header { }	The one element with id="header".
Universal	* { }	All elements.
Grouping	h1, h2, p { }	Several selectors sharing the same rule.
Descendant / Child	ul li { } / ul > li { }	Any li inside ul / only direct children.
Attribute	input[type="text"] { }	Elements having that attribute value.

Exam tip - key points to remember

- CSS = Cascading Style Sheets; separates presentation from content. Three types: **inline** (style attribute), **internal** (<style> in head), **external** (<link> to .css file).
- Priority: inline > internal/external (later wins) > browser default. External CSS is best for whole sites; inline is worst for maintenance.

Q2 (b) [10 Marks, CO2]

Write HTML and CSS code for the following: (i) Set the background color for the active link states to 'yellow'. (ii) Set the list style for unordered lists to 'square'. (iii) Set 'paper.gif' as the background image of the page.

1. Complete solution (HTML with internal CSS)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS - Question 2(b)</title>
  <style>
    /* (i) background colour of a link while it is being clicked (active state) */
    a:active {
      background-color: yellow;
    }

    /* (ii) unordered lists use square bullets */
    ul {
      list-style-type: square;
    }

    /* (iii) paper.gif as the background image of the whole page */
    body {
      background-image: url("paper.gif");
    }
  </style>
</head>
<body>
  <h2>Useful links</h2>

```

```
<ul>
  <li><a href="https://www.w3.org">W3C</a></li>
  <li><a href="https://developer.mozilla.org">MDN Web Docs</a></li>
  <li><a href="https://www.wikipedia.org">Wikipedia</a></li>
</ul>
<p>Press and hold the mouse button on a link to see the yellow active background.</p>
</body>
</html>
```

2. Explanation of each part

(i) Background colour of active links - a:active

A link can be in four states, which are styled with **pseudo-classes**: `a:link` (unvisited), `a:visited` (visited), `a:hover` (mouse over) and `a:active` (the moment it is clicked, while the mouse button is pressed). The property `background-color` sets the background colour of the element. So `a:active { background-color: yellow; }` makes the link's background yellow only while it is being activated. If the paper asks for the text colour to become yellow instead, use the `color` property.

Order rule (LoVe HAtE): when all four are used, write them in the order **a:link, a:visited, a:hover, a:active**; otherwise a later rule may override the earlier ones.

(ii) Square bullets - list-style-type

The `list-style-type` property selects the marker of a list. For unordered lists (`<ul>`) the values are **disc** (default), **circle**, **square** and **none**. The selector `ul` affects all unordered lists of the page; to apply it to only one list a class can be used (`ul.sq { list-style-type: square; }`).

Property	Values	Effect
<code>list-style-type (ul)</code>	<code>disc circle square none</code>	Shape of the bullet.
<code>list-style-type (ol)</code>	<code>decimal lower-alpha upper-alpha lower-roman upper-roman</code>	Numbering style of ordered lists.
<code>list-style-image</code>	<code>url("bullet.png")</code>	Uses an image as the bullet.
<code>list-style-position</code>	<code>outside inside</code>	Marker outside (default) or inside the text block.
<code>list-style</code>	<code>square inside</code>	Shorthand for type, position and image.

(iii) Background image - background-image

The `background-image` property places an image behind the element. Applied to `body` it becomes the background of the whole page. The image file is given with the `url()` function; the path is relative to the CSS file (or to the HTML file for internal CSS). By default the image **repeats** horizontally and vertically to fill the page (tiling), which suits a small texture like `paper.gif`. Related properties give more control:

Property	Common values	Meaning
<code>background-color</code>	<code>yellow, #ffff00, rgb(255,255,0)</code>	Solid colour behind the element (also a fallback if the image fails to load).
<code>background-image</code>	<code>url("paper.gif")</code>	The image to be used.
<code>background-repeat</code>	<code>repeat repeat-x repeat-y no-repeat</code>	Whether and how the image is tiled.
<code>background-position</code>	<code>center top right 20px 40px</code>	Starting position of the image.
<code>background-attachment</code>	<code>scroll fixed</code>	Image scrolls with the page or stays fixed.
<code>background-size</code>	<code>auto cover contain 100px 50px</code>	Size of the background image.

Property	Common values	Meaning
background	url("paper.gif") repeat fixed	Shorthand for all the background properties.

3. Same solution using an external style sheet

```
/* style.css */
a:active { background-color: yellow; }
ul      { list-style-type: square; }
body    { background-image: url("paper.gif"); background-color: #fdf6e3; }

<!-- in the HTML file -->
<link rel="stylesheet" href="style.css">
```

Expected output: the page appears with the paper texture behind all content; the three links are shown as a bulleted list with **small square** bullets; when the user presses the mouse button on a link, its background flashes yellow before the new page opens.

Exam tip - key points to remember

- a:active { background-color: yellow; } ul { list-style-type: square; } body { background-image: url("paper.gif"); }
- Order of link pseudo-classes: link, visited, hover, active. Image path must be correct or the background will not appear.

Question 3

10 Marks | Attempt any ONE sub-question in the exam | CO2 | Both options are answered below for revision

Q3 (a) [10 Marks, CO2]

Write CSS code that define five classes of paragraph with different background, color, margins, padding and border style.

1. Concept - CSS classes

A **class selector** is written with a **dot (.)** followed by the class name, e.g. `.note`. It can be applied to **any number of elements** by adding the attribute `class="note"` to the tag. To limit the rule to paragraphs only, the element name is written before the dot: `p.note`. An element can also carry several classes at once (`class="note big"`). Class names are case-sensitive and must not start with a digit. In contrast to an **id** (`#name`) which must be unique in a page, a class is meant for reuse.

2. CSS code - five paragraph classes

```
/* ----- Class 1 : note ----- */
p.note {
    background-color: #FFF8DC;          /* cornsilk / light yellow */
    color: #5A4A00;                    /* dark brown text */
    margin: 10px 20px;                 /* top-bottom 10px, left-right 20px */
    padding: 12px;                     /* same on all four sides */
    border: 2px solid #E0B84A;
}
/* ----- Class 2 : warning ----- */
p.warning {
    background-color: #FDE2E2;
    color: #8A1F1F;
    margin: 15px 40px;
    padding: 15px 20px;                /* top-bottom 15px, left-right 20px */
    border: 3px dashed #D9534F;
}
/* ----- Class 3 : success ----- */
p.success {
    background-color: #E3F4E6;
    color: #1E6B34;
    margin: 10px 60px;
    padding: 10px 25px;
    border: 2px dotted #3E8B55;
}
/* ----- Class 4 : info ----- */
p.info {
    background-color: #E1E9FB;
    color: #114A7B;
    margin: 20px;
    padding: 20px 10px 20px 10px;      /* top right bottom left */
    border: 5px double #2C6CA8;
}
/* ----- Class 5 : quote ----- */
p.quote {
    background-color: #F1F1F1;
    color: #333333;
    font-style: italic;
    margin: 10px 30px 10px 30px;
    padding: 10px 15px;
    border: 4px groove #888888;
    border-left: 8px solid #555555;    /* overrides only the left border */
}
```

3. HTML using the classes

```
<!DOCTYPE html>
<html>
<head>
  <title>Paragraph classes</title>
  <link rel="stylesheet" href="style.css"          <!-- the CSS above, or place it in <style> -->
</head>
<body>
  <p class="note">Note: submit the assignment before Friday.</p>
  <p class="warning">Warning: do not delete system files.</p>
  <p class="success">Success: your form was submitted.</p>
  <p class="info">Info: the library is open from 9 AM to 5 PM.</p>
  <p class="quote">"Simplicity is the ultimate sophistication."</p>
</body>
</html>
```

Summary of the five classes

Class	Background	Text colour	Margin	Padding	Border style
.note	#FFF8DC (light yellow)	#5A4A00	10px 20px	12px	2px solid
.warning	#FDE2E2 (light red)	#8A1F1F	15px 40px	15px 20px	3px dashed
.success	#E3F4E6 (light green)	#1E6B34	10px 60px	10px 25px	2px dotted
.info	#E1EFFF (light blue)	#114A7B	20px	20px 10px	5px double
.quote	#F1F1F1 (light grey)	#333333	10px 30px	10px 15px	4px groove , thick left

Approximate output in the browser

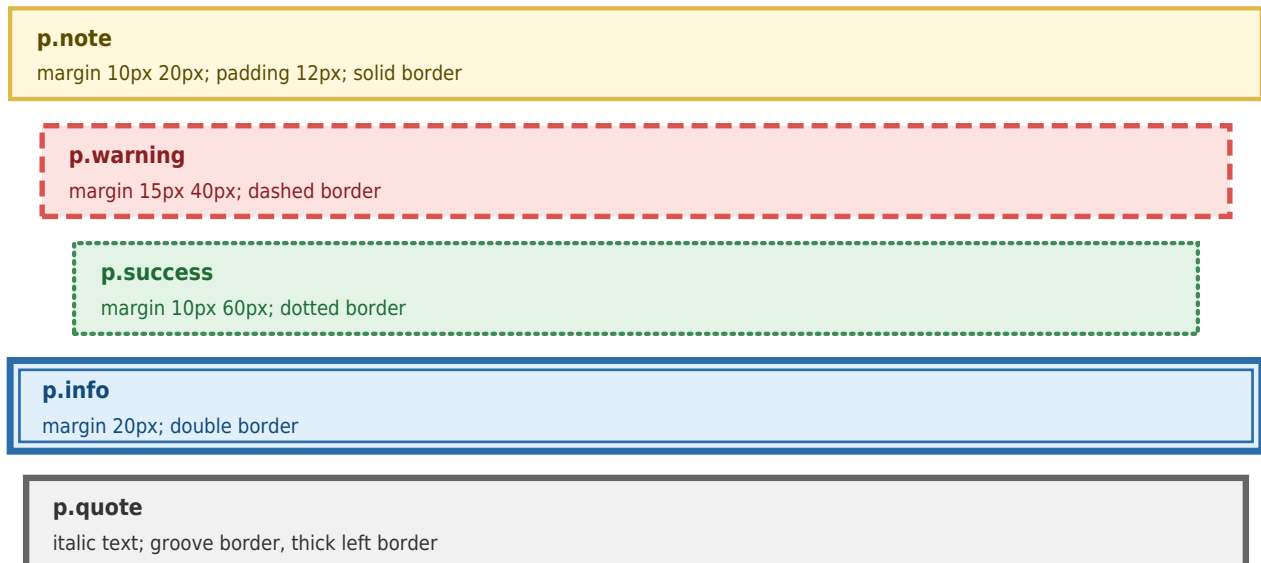


Fig. 2 - Sketch of the five styled paragraphs (differences in margin, padding, colour and border style)

4. Properties used

Property	Function	Example
background-color	Fills the content and padding area with a colour.	background-color: #FFF8DC;
color	Colour of the text.	color: #5A4A00;
margin	Outer space between the border and neighbouring elements (transparent).	margin: 10px 20px;
padding	Inner space between the content and the border (shows the background).	padding: 15px 20px;

Property	Function	Example
border	Shorthand: width, style, colour. Individual sides: border-left, border-top ...	border: 3px dashed red;

Shorthand rules for margin and padding

Number of values	Meaning	Example
1 value	All four sides.	padding: 12px;
2 values	top-bottom, then left-right.	margin: 10px 20px;
3 values	top, then left-right, then bottom.	margin: 5px 10px 15px;
4 values	top, right, bottom, left (clockwise).	padding: 20px 10px 20px 10px;

Values of border-style

Value	Look	Value	Look
solid	Single continuous line.	groove	3-D grooved border (depends on colour).
dashed	Series of short dashes.	ridge	3-D ridged border (opposite of groove).
dotted	Series of dots.	inset / outset	3-D border that looks embedded / raised.
double	Two parallel lines (width must be at least 3px).	none / hidden	No border.

Exam tip - key points to remember

- Class selector = **.classname** (or p.classname); used in HTML as class="classname". Write five rules, each with background-color, color, margin, padding, border.
- Margin = outside the border; padding = inside the border. Use a different border-style in each class: solid, dashed, dotted, double, groove.

Q3 (b) [10 Marks, CO2]

What are CSS pseudo-classes? Explain any four pseudo-classes with suitable example.

1. What is a pseudo-class?

A **pseudo-class** is a keyword added to a selector, preceded by a **single colon (:)**, that specifies a **special state** of the selected element or its position in the document. It lets the designer style an element depending on something that is not present as a class or id in the HTML - for example when the mouse is over it, when it has keyboard focus, when a link has been visited, or when it is the first child of its parent. The browser adds and removes the pseudo-class automatically.

```
selector:pseudo-class {
    property: value;
}
a:hover      { color: red; }          /* link while the mouse pointer is over it */
li:first-child { font-weight: bold; } /* the first item of every list          */
```

2. Four pseudo-classes explained with examples

(i) :hover - when the mouse pointer is over the element

Used for buttons, links, menus and table rows to give visual feedback. The style disappears when the pointer moves away.

```
a:hover { color: red; text-decoration: underline; }
button:hover { background-color: #2C6CA8; color: white; }
```

(ii) :focus - when the element has keyboard focus

An element gets focus when the user clicks or tabs into it (form fields, links, buttons). It helps the user to see which input field is active.

```
input:focus { background-color: #FFFFCC; border: 2px solid blue; outline: none; }
```

(iii) :first-child - the first child of its parent

Selects an element only if it is the first child of its parent. Similarly, :last-child selects the last one and :only-child an element without siblings.

```
li:first-child { font-weight: bold; color: green; }      /* first item of every list */
p:first-child { font-size: 20px; }
```

(iv) :nth-child(n) - the n-th child by position

Selects children by position. The value can be a number (:nth-child(3)), the keywords **odd** or **even**, or a formula such as **2n+1**. It is commonly used to make **zebra-striped tables**.

```
tr:nth-child(even) { background-color: #EAF1F8; }      /* even rows shaded */
tr:nth-child(odd) { background-color: #FFFFFF; }
li:nth-child(3) { color: red; }                       /* only the third item */
```

Complete example

```
<!DOCTYPE html>
<html>
<head>
<style>
  a:link { color: blue; } /* unvisited link */
  a:visited { color: purple; } /* visited link */
  a:hover { color: red; text-decoration: underline; }
  a:active { background-color: yellow; } /* while being clicked */
  input:focus { background-color: #FFFFCC; border: 2px solid blue; }
  li:first-child { font-weight: bold; color: green; }
  tr:nth-child(even) { background-color: #EAF1F8; }
</style>
</head>
<body>
  <a href="https://www.w3.org">Visit W3C</a>
  <p>Name: <input type="text" placeholder="click here to see :focus"></p>
  <ul><li>First (bold, green)</li><li>Second</li><li>Third</li></ul>
  <table border="1">
    <tr><td>Row 1</td></tr><tr><td>Row 2 (shaded)</td></tr>
    <tr><td>Row 3</td></tr><tr><td>Row 4 (shaded)</td></tr>
  </table>
</body>
</html>
```


3. Other important pseudo-classes

Pseudo-class	Matches when	Example
:link	A link that has not been visited.	a:link { color: blue; }
:visited	A link that has been visited.	a:visited { color: purple; }
:active	The element is being activated (mouse button pressed).	a:active { background: yellow; }
:checked	Checked checkbox or radio button, selected option.	input:checked + label { color: green; }
:disabled / :enabled	Form control that is disabled / enabled.	input:disabled { background: #ddd; }
:required / :optional	Form field with / without the required attribute.	input:required { border-color: red; }
:last-child	Last child of its parent.	li:last-child { border: none; }
:first-of-type	First element of its type among siblings.	p:first-of-type { font-size: 18px; }
:not(selector)	Elements that do not match the selector.	p:not(.note) { color: gray; }
:empty	Element with no children or text.	p:empty { display: none; }
:target	Element whose id matches the URL fragment.	#sec1:target { background: #ffc; }
:root	The root element (html); used for CSS variables.	:root { --main: navy; }

4. Pseudo-class vs pseudo-element

Basis	Pseudo-class	Pseudo-element
Syntax	Single colon: a:hover	Double colon (CSS3): p::first-line
Purpose	Selects an element in a particular state or position .	Selects or creates a part of an element.
Examples	:hover, :focus, :first-child, :nth-child()	::before, ::after, ::first-letter, ::first-line, ::selection
Can be combined?	Yes, several pseudo-classes can be chained: a:hover:not(.x)	Only one pseudo-element per selector, at the end.

Exam tip - key points to remember

- Pseudo-class = **selector:state** with one colon. Four to write: **:hover, :focus, :first-child, :nth-child()** (or :link, :visited, :active).
- Link order: link, visited, hover, active. Pseudo-elements use double colon (: : before).

Question 4

10 Marks | Attempt any ONE sub-question in the exam | CO1 | Both options are answered below for revision

Q4 (a) [10 Marks, CO1]

(i) Explain `<frame>` tag with its attributes. (ii) Explain the attributes of `<input>` tag with an example.

(i) The `<frame>` tag

Frames divide the browser window into **several independent rectangular sections**, and each section (frame) displays a **separate HTML document**. A common use is a fixed menu in one frame and the selected content in another, so that clicking a menu link changes only the content frame. Frames are created with three tags: `<frameset>` (defines the layout as rows and/or columns and *replaces* the `<body>` tag), `<frame>` (defines one frame and the document that goes into it) and `<noframes>` (text for browsers that do not support frames).

The `<frame>` tag is an **empty tag** (no closing tag); it must be written **inside a `<frameset>`**. **Note:** `<frameset>`, `<frame>` and `<noframes>` are **obsolete in HTML5** (they belong to HTML 4.01 Frameset). Modern pages use `<iframe>` and CSS layout instead - see below. The tags are still asked in examinations.

Attributes of `<frame>`

Attribute	Values	Purpose
src	URL	Document to be displayed in the frame (required).
name	text	Name of the frame; used as the target of links and forms so that a link can load a page into this frame.
frameborder	1 0	Shows (1) or hides (0) the border around the frame.
scrolling	yes no auto	Scroll bars always / never / only when needed.
noresize	noresize	Stops the user from resizing the frame by dragging its border.
marginwidth	pixels	Left and right margin inside the frame.
marginheight	pixels	Top and bottom margin inside the frame.
longdesc	URL	Link to a long description of the frame's content.

Attributes of `<frameset>` (layout)

Attribute	Purpose	Example
rows	Divides the window into horizontal frames; sizes in pixels, % or * (remaining space).	rows="25%, 75%"
cols	Divides the window into vertical frames.	cols="200, *"
border, bordercolor, frameborder, framespacing	Width, colour, visibility and spacing of the frame borders.	border="2" bordercolor="blue"

Example - header, menu and content frames

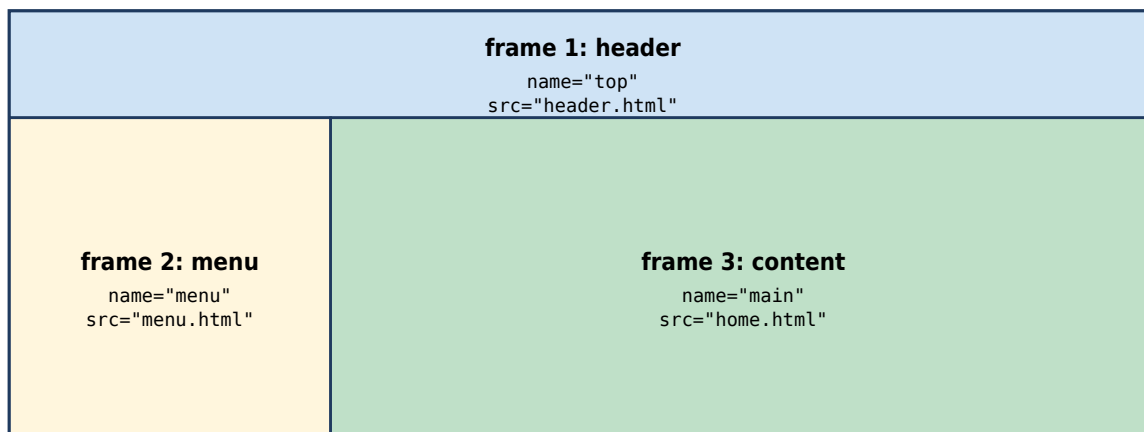


Fig. 3 - Layout produced by the nested framesets below

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"
    "http://www.w3.org/TR/html4/frameset.dtd">
<html>
<head><title>Frames example</title></head>

<frameset rows="25%,75%">                                <!-- two horizontal parts -->
  <frame src="header.html" name="top" scrolling="no" noresize>
  <frameset cols="28%,72%">                                <!-- lower part: two columns -->
    <frame src="menu.html" name="menu">
    <frame src="home.html" name="main">
  </frameset>
</frameset>
<noframes>
  <body>Your browser does not support frames.</body>
</noframes>
</frameset>
</html>
<!-- inside menu.html : the link loads the page into the frame called "main" -->
<a href="notes.html" target="main">Notes</a>
```

Advantages, disadvantages and the modern alternative

Advantages of frames	Disadvantages of frames
A menu or logo stays fixed while only the content changes; the fixed part is loaded once.	Bookmarking and printing a page is difficult: the URL shows only the frameset, not the page in the frame.
Each frame can scroll separately.	Poor for search engines and screen readers; small screens/mobile are not supported well.
Different documents can be shown side by side.	Extra HTTP requests and slower loading; deprecated in HTML5 and removed from browsers' standards.

Replacement - inline frame <iframe>: embeds a document inside a normal page (still valid in HTML5), e.g. maps, videos, other sites. Attributes: src, name, width, height, title, srcdoc, sandbox, loading, allowfullscreen.

```
<iframe src="https://www.example.com" name="view" width="600" height="300"
    title="Example site"></iframe>
```

(ii) Attributes of the <input> tag

The <input> tag is the **most important form element**. It creates an input control (text box, password box, check box, radio button, button, file selector, date picker...) where the user can enter data. It is an **empty tag** and its behaviour is decided by its **attributes**, mainly type.

Main attributes of <input>

Attribute	Purpose	Example
type	Kind of control: text, password, email, number, date, checkbox, radio, submit, reset, button, file, hidden, range, color, url, tel, search, image. Default is text.	type="email"
name	Name of the field; the name/value pair is sent to the server on submission.	name="user"
value	Initial value of the field, or the caption of a button; the value sent for radio/checkbox.	value="Submit"
id	Unique identifier; used to link with <label for>, CSS and JavaScript.	id="u1"
placeholder	Hint text shown inside an empty field.	placeholder="Enter name"
size / maxlength / minlength	Visible width in characters / maximum / minimum number of characters allowed.	maxlength="20"
min / max / step	Limits and increment for number, range and date fields.	min="1" max="10" step="1"
required	Field must be filled before the form can be submitted (HTML5 validation).	required
readonly / disabled	Value cannot be edited, but is sent / control is inactive and not sent.	readonly
checked	Pre-selects a checkbox or radio button.	checked
pattern	Regular expression that the value must match.	pattern="[0-9]{10}"
autofocus / autocomplete	Focus the field on page load / allow the browser to fill in previous values (on off).	autofocus autocomplete="off"
multiple / accept	Allow several values or files / file types accepted by a file field.	accept="image/*"
list / form	Connects to a <datalist> of suggestions / to a form by its id.	list="cities"

Values of the type attribute

type	Control created	type	Control created
text	Single-line text box.	radio	Choose one option from a group (same name).
password	Text hidden as dots.	checkbox	Choose zero or more options.
email / url / tel	Text with format validation.	file	Browse and upload a file.
number / range	Numeric field / slider.	date / time	Date or time picker.
submit / reset	Send the form / clear it.	hidden	Invisible field carrying data.
button	Push button for scripts.	color / search	Colour picker / search box.

Example

```
<form action="register.php" method="post">
  <p>Name:      <input type="text" name="fname" id="fname" size="25" maxlength="30"
                  placeholder="Enter full name" required autofocus></p>
  <p>Password: <input type="password" name="pwd" minlength="6" required></p>
  <p>Email:    <input type="email" name="mail" placeholder="abc@example.com"></p>
  <p>Age:      <input type="number" name="age" min="18" max="60" value="21"></p>
  <p>Mobile:   <input type="text" name="mob" pattern="[0-9]{10}" title="10 digit number"></p>
  <p>Gender:   <input type="radio" name="gender" value="M" checked> Male
              <input type="radio" name="gender" value="F"> Female</p>
  <p>Photo:   <input type="file" name="photo" accept="image/*"></p>
  <p><input type="submit" value="Register"> <input type="reset" value="Clear"></p>
</form>
```

Exam tip - key points to remember

- `<frame>` = empty tag inside `<frameset rows|cols>`; attributes `src`, `name`, `scrolling`, `noresize`, `frameborder`, `marginwidth/height`. Obsolete in HTML5, replaced by `<iframe>`.
- `<input>` = empty tag; the **type** attribute decides the control; also `name`, `value`, `size`, `maxlength`, `placeholder`, `required`, `readonly`, `disabled`, `checked`, `min/max`, `pattern`.

Q4 (b) [10 Marks, CO1]

What is a form? What are the major attributes of the form? Explain any four form components with example.

1. What is a form?

An **HTML form** is a section of a web page, created with the `<form>` tag, that **collects data from the user** and **sends it to a server** for processing. It contains form controls such as text fields, radio buttons, check boxes, drop-down lists and buttons. Uses: login and registration, online examination, search, feedback, online shopping and bank transactions. The typical processing cycle is shown below.

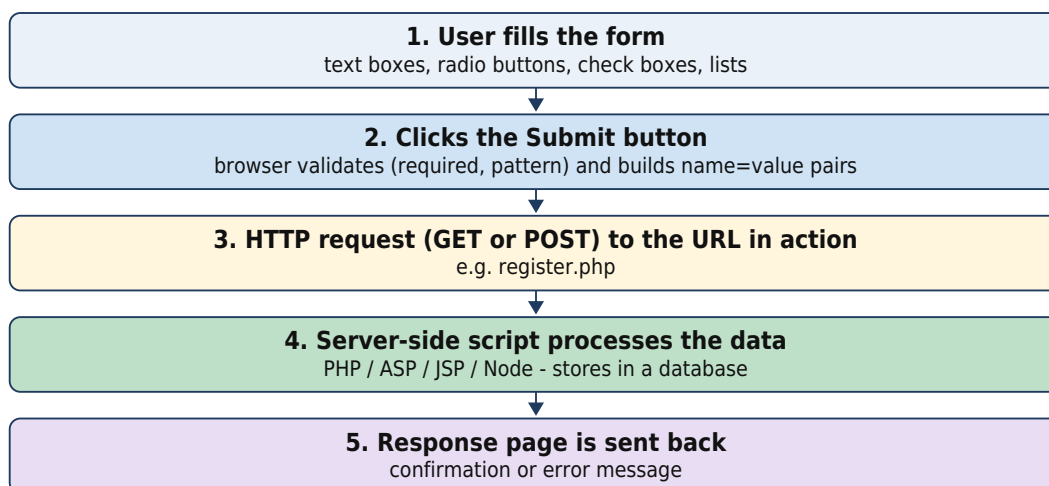


Fig. 4 - Working of a form

2. Major attributes of the `<form>` tag

Attribute	Purpose	Values / example
action	URL of the program (server script) that receives and processes the form data. If omitted, the same page is used.	action="save.php"
method	HTTP method used to send the data.	get post

Attribute	Purpose	Values / example
enctype	How the data is encoded when method is post; multipart/form-data is needed for file upload.	application/x-www-form-urlencoded, multipart/form-data, text/plain
target	Where the response is displayed.	_self, _blank, _parent, _top
name / id	Name / unique id of the form (used by scripts).	name="regform"
autocomplete	Whether the browser may suggest earlier values.	on off
novalidate	Turns off the browser's built-in validation on submit.	novalidate
accept-charset	Character encodings accepted by the server.	accept-charset="UTF-8"

GET vs POST method

Basis	GET	POST
Where data goes	Appended to the URL as a query string: page.php?name=Ravi&age=21	Sent in the body of the HTTP request; not visible in the URL.
Security	Less secure - data visible in URL, history and logs; never use for passwords.	More secure (still needs HTTPS for real protection).
Size limit	Limited by URL length (about 2000 characters).	No practical limit; suitable for large data and files.
Bookmark / cache	Can be bookmarked and cached.	Cannot be bookmarked; not cached.
Use	Searching, filtering, retrieving data.	Login, registration, uploading, updating data.

3. Form components (controls) with examples

(i) Text field - single-line input

Created with `<input type="text">` (also password, email, number). Used for names, user ids, passwords. A `<label>` describes the field; clicking the label focuses the field.

```
<label for="uname">User name:</label>
<input type="text" id="uname" name="uname" size="20" maxlength="25" placeholder="Enter name" required>
<input type="password" name="pwd" placeholder="Password">
```

(ii) Radio button - choose only ONE option from a group

Created with `<input type="radio">`. All radio buttons of a group must have the **same name**; the value of the selected one is sent. Use checked for a default choice.

```
Gender:
<input type="radio" name="gender" value="male" checked> Male
<input type="radio" name="gender" value="female"> Female
<input type="radio" name="gender" value="other"> Other
```

(iii) Checkbox - choose ZERO or MORE options

Created with `<input type="checkbox">`. Each box is independent and normally has its own value; every ticked box is sent to the server.

```
Hobbies:
<input type="checkbox" name="hobby" value="reading"> Reading
<input type="checkbox" name="hobby" value="music" checked> Music
<input type="checkbox" name="hobby" value="sports"> Sports
```

(iv) Drop-down list (select) - choose from a list

Created with `<select>` containing `<option>` items (grouped by `<optgroup>`). Attributes: name, size (rows visible), multiple (select more than one), selected (default option).

```
Course:
<select name="course">
  <option value="mca" selected>MCA</option>
  <option value="bca">BCA</option>
  <option value="btech">B.Tech</option>
</select>
```

(v) Textarea - multi-line text

Created with `<textarea>` for comments, address, feedback. Attributes: rows, cols, name, maxlength, placeholder. The default text is written between the tags.

```
<textarea name="comments" rows="4" cols="40" placeholder="Write your feedback here"></textarea>
```

(vi) Buttons

`<input type="submit">` sends the form; `<input type="reset">` clears all fields; `<input type="button">` runs a script; `<button>` is a more flexible button that may hold text and images.

4. Complete form

```
<form action="register.php" method="post" name="regform">
  <fieldset>
    <legend>Student Registration</legend>
    <p><label>Name: <input type="text" name="sname" required></label></p>
    <p>Gender: <input type="radio" name="g" value="M" checked> Male
      <input type="radio" name="g" value="F"> Female</p>
    <p>Hobbies: <input type="checkbox" name="h" value="r"> Reading
      <input type="checkbox" name="h" value="m"> Music</p>
    <p>Course: <select name="course"><option>MCA</option><option>BCA</option></select></p>
    <p>Address:<br><textarea name="addr" rows="3" cols="35"></textarea></p>
    <input type="submit" value="Register"> <input type="reset" value="Reset">
  </fieldset>
</form>
```

Exam tip - key points to remember

- Form = `<form action method>` collecting user data for a server. Major attributes: action, method (get/post), enctype, target, name, autocomplete, novalidate.
- Any four components: text field, radio button (same name), checkbox, select list, textarea, buttons - each with a code line.

Question 5

10 Marks | Attempt any ONE sub-question in the exam | CO1 | Both options are answered below for revision

Q5 (a) [10 Marks, CO1]

Write HTML code that displays three hyperlinks to different websites. The websites should open in a new window when the user clicks on the hyperlinks.

1. Concept - hyperlink and the <a> tag

A **hyperlink** is a clickable element (text or image) that takes the user to another page, another part of the same page, a file or an e-mail program. It is created with the **anchor tag** <a>. The destination is given in the **href** (hypertext reference) attribute and the clickable text is written between the opening and closing tags. To open the destination in a **new window (tab)** the attribute **target="_blank"** is added.

```
<a href="URL" target="_blank">Link text</a>
```

2. HTML code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Hyperlinks to three websites</title>
</head>
<body>
  <h2>Popular websites (each link opens in a new window)</h2>
  <ul>
    <li><a href="https://www.google.com" target="_blank" rel="noopener noreferrer"
      title="Search engine">Google</a></li>

    <li><a href="https://www.wikipedia.org" target="_blank" rel="noopener noreferrer"
      title="Free encyclopedia">Wikipedia</a></li>
    <li><a href="https://www.w3schools.com" target="_blank" rel="noopener noreferrer"
      title="Web tutorials">W3Schools</a></li>
  </ul>
</body>
</html>
```

Output: the page shows a heading and a bulleted list with three blue underlined links - *Google*, *Wikipedia* and *W3Schools*. Hovering shows the title as a tool-tip. Clicking any link opens that website in a **new browser tab/window** and the original page remains open.

3. Explanation

- <a> - anchor tag that creates the link; href gives the absolute URL of the website (including https://).
- target="_blank" - opens the linked document in a new browsing context. Without it the link opens in the same window (_self, default).
- rel="noopener noreferrer" - a security measure for links that open a new window: the new page cannot access the original page through window.opener (protection against 'tabnabbing') and the referrer is not passed.
- title - tool-tip text; the text between <a> and is the visible link. By default a link is **blue and underlined** if unvisited, **purple** if visited and **red** while active.

4. Values of the target attribute

Value	Where the linked page opens
_self	In the same window/tab (default).
_blank	In a new window or tab.
_parent	In the parent frame (of a frameset).
_top	In the full window, replacing all frames.
framename	In the frame or named window having that name; links with the same name reuse the same window.

5. More about hyperlinks

Attributes of the <a> tag

Attribute	Purpose	Example
href	Destination URL or #id of an anchor.	href="notes.html"
target	Where to open the link.	target="_blank"
rel	Relationship between the pages (noopener, noreferrer, nofollow).	rel="nofollow"
download	Downloads the file instead of opening it.	download="form.pdf"
title	Tool-tip text.	title="Go to W3C"
hreflang / type	Language and MIME type of the linked document.	hreflang="en"

Types of links

Type	Example	Description
Absolute (external) link	<code></code>	Complete URL to another website.
Relative (internal) link	<code></code>	Path to a page of the same site, relative to the current file.
Bookmark (same page)	<code></code> and <code><h2 id="sec2"></code>	Jumps to the element with that id in the same page.
E-mail link	<code></code>	Opens the e-mail program with the address filled in.
Telephone link	<code></code>	Dials the number on phones.
Image link	<code></code>	The image is clickable.

Opening a real pop-up window (with JavaScript)

Browsers usually open `target="_blank"` in a new *tab*. To force a separate window of a given size, JavaScript `window.open()` is used:

```
<a href="https://www.google.com"
  onclick="window.open(this.href, 'win1', 'width=800,height=600'); return false;">
  Google (pop-up window)
</a>

<!-- make ALL links of a page open in a new window by writing this once in <head> -->
<base target="_blank">
```

Exam tip - key points to remember

- Three `<a href="..." target="_blank">` tags = answer; add `rel="noopener noreferrer"` for security and mention that `_blank` opens a new window/tab.
- Explain `href` (destination), `target` (where to open) and the link text between `<a>` and `</a>`.

Q5 (b) [10 Marks, CO1]

Explain CSS box model concept with the help of block diagram.

1. What is the CSS box model?

In CSS every HTML element is treated as a **rectangular box**. The **box model** describes how the size and the space around this box are calculated. It is the basis of all page layout: it decides how much space an element occupies, how far it stays from its neighbours and how its border and background look. The box consists of four layers, from the inside to the outside: **Content** → **Padding** → **Border** → **Margin**.

2. Block diagram of the box model

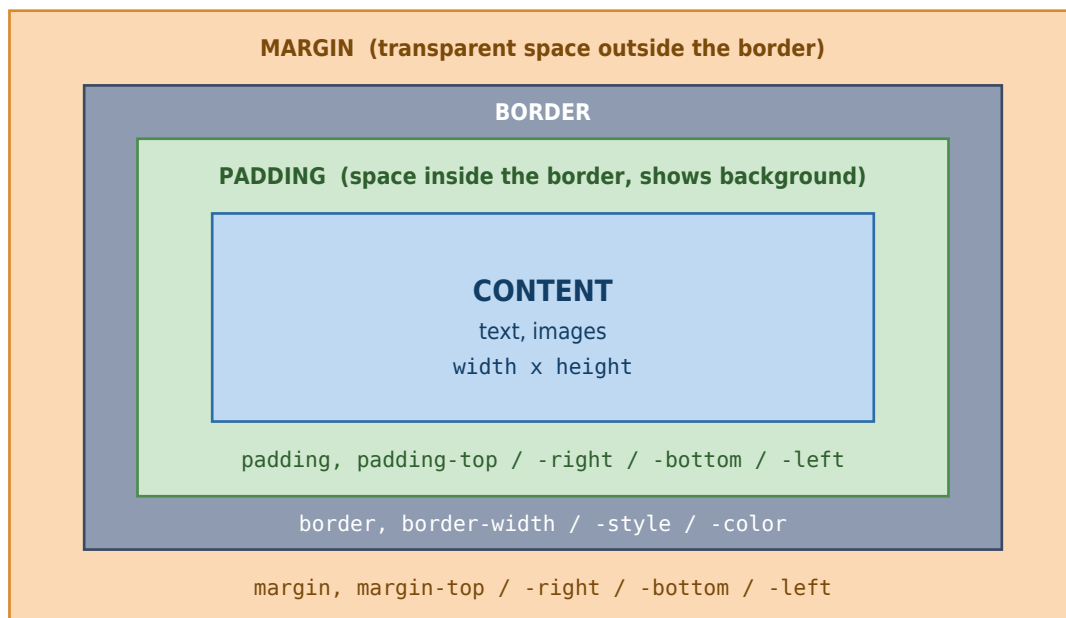


Fig. 5 - CSS box model: content, padding, border and margin

3. Parts of the box model

Part	Description	Properties	Background applies?
Content	The actual text, image or other content of the element. Its size is set by width and height.	width, height, min-/max-width, overflow	Yes
Padding	Transparent space between the content and the border ; it takes the element's background colour or image.	padding, padding-top, -right, -bottom, -left	Yes
Border	A line around the padding and content; has width, style and colour.	border, border-width, border-style, border-color, border-radius	Border colour itself
Margin	Transparent space outside the border ; separates the element from other elements. Never coloured.	margin, margin-top, -right, -bottom, -left	No (always transparent)

4. Example

```
<style>
  div.box {
    width: 300px;           /* content width */
    height: 100px;          /* content height */
    padding: 20px;          /* space inside the border */
    border: 5px solid #1F3A5F; /* the border */
    margin: 10px;           /* space outside the border */
    background-color: #EAF1F8;
  }
</style>
<div class="box">This is the content of the box.</div>
```

Calculating the total size (default box-sizing: content-box)

Measure	Calculation	Result
Total width of the box (border edge to border edge)	300 (content) + 2 x 20 (padding) + 2 x 5 (border)	350 px
Total width including margin (space occupied in the layout)	350 + 2 x 10 (margin)	370 px
Total height of the box	100 (content) + 2 x 20 (padding) + 2 x 5 (border)	150 px
Total height including margin	150 + 2 x 10 (margin)	170 px

Formula: total width = width + left padding + right padding + left border + right border + left margin + right margin. A similar formula is used for the height.

5. The box-sizing property

By default (content-box) the width and height set the size of the *content only*; padding and border are added on top, which makes layouts hard to calculate. With box-sizing: border-box; the width and height *include* padding and border, so the box keeps the size that was specified.

Basis	content-box (default)	border-box
Meaning of width	Width of the content area only.	Width of content + padding + border.
Example: width 300px, padding 20px, border 5px	Visible box is 350px wide.	Visible box stays 300px wide; content shrinks to 250px.
Advantage	Simple definition of the content size.	Easier and more predictable layouts; widely used: * { box-sizing: border-box; }

6. Important points

- Margin collapsing:** when the vertical margins of two adjacent block elements meet, they collapse into **one margin equal to the larger** of the two (30px and 20px give 30px, not 50px). Horizontal margins never collapse.
- Centering:** a block with a fixed width is centred horizontally using `margin: 0 auto;`.
- Outline** (outline) is drawn outside the border but is **not part of the box model** and takes no space.
- Block vs inline:** block elements (div, p, h1) use the full box model and start on a new line; inline elements (span, a) ignore width/height and vertical margins.

Exam tip - key points to remember

- Draw four nested rectangles: **content, padding, border, margin** (inside to outside) and label them; add the total-width formula and one numeric example (370 px).
- Padding is inside the border and coloured by the background; margin is outside the border and transparent. Mention box-sizing.

Quick Revision Sheet

Cheat-sheets and practice questions for the whole HTML/CSS part of the syllabus

1. Important HTML tags

Group	Tags	Remember
Structure	html, head, title, body, meta, link, style, script	Metadata goes in head; only body content is displayed.
Text	h1-h6, p, br, hr, b, i, u, strong, em, sub, sup, pre, blockquote	br and hr are empty tags.
Links and images	a, img, map, area	a: href, target; img: src, alt, width, height, usemap.
Lists	ul, ol, li, dl, dt, dd	ul = bullets, ol = numbers (type, start), dl = definitions.
Tables	table, tr, th, td, caption, thead, tbody, tfoot	Attributes: border, colspan, rowspan, cellpadding, cellspacing.
Forms	form, input, select, option, textarea, button, label, fieldset, legend	action, method; input type decides control.
Frames	frameset, frame, noframes, iframe	Frames obsolete in HTML5; iframe still valid.
Semantic HTML5	header, nav, section, article, aside, footer, main, figure	Give meaning to the layout; better SEO and accessibility.
Multimedia	audio, video, source, canvas, svg	Attributes: controls, autoplay, loop.

2. Important CSS properties

Category	Properties
Text and font	color, font-family, font-size, font-weight, font-style, text-align, text-decoration, text-transform, line-height, letter-spacing
Background	background-color, background-image, background-repeat, background-position, background-attachment, background-size
Box model	width, height, margin, padding, border, border-radius, box-sizing, overflow, outline
Lists	list-style-type, list-style-image, list-style-position
Display and position	display (block, inline, inline-block, none, flex, grid), position (static, relative, absolute, fixed, sticky), top, left, z-index, float, clear
Effects	box-shadow, text-shadow, opacity, transition, transform, animation, cursor

3. Selector types and specificity

Selector	Example	Specificity weight
Inline style	style="..."	1000 (highest)
ID	#menu	100
Class, attribute, pseudo-class	.note, [type], :hover	10

Selector	Example	Specificity weight
Element, pseudo-element (universal * = 0)	p, ::before	1

4. Practice questions from the same syllabus

1. Explain the basic structure of an HTML document. Differentiate between HTML and XHTML, and between HTML 4 and HTML5.
2. Write HTML code for a table with row span and column span; explain the attributes of the <table> tag.
3. Explain ordered, unordered and definition lists with examples. How are nested lists created?
4. Explain the semantic tags of HTML5 with a sample page layout (header, nav, section, aside, footer).
5. Difference between class selector and id selector. What is CSS specificity and the !important rule?
6. Explain CSS positioning (static, relative, absolute, fixed, sticky) and the float and clear properties.
7. Write CSS code to create a horizontal navigation menu from an unordered list, with hover effect.
8. Explain CSS units (px, em, rem, %, vw, vh) and colour models (name, hex, rgb, hsl).
9. What are media queries? Write CSS to change the layout for screens smaller than 600 px.
10. Introduction to JavaScript: variables, data types, operators, functions, events and simple form validation.